



Our Github

ECE 455 - GPU Algorithm and System Design

CudInference

Custom GPU-Accelerated Neural Network Inference Engine

Damion Huppert
Irish Damaraju
Chris Vanden Boom
Aiden Carney



Motivation

→ Running modern **AI inference** (CNNs, Transformers, MLPs, etc.) purely on the CPU often fails latency and throughput targets. Our objective is to accelerate inference by offloading compute-heavy operations to a **CUDA**-capable GPU, while minimizing data-transfer overhead and preserving correctness. This enables efficient local execution of ML models, even on resource-constrained systems.

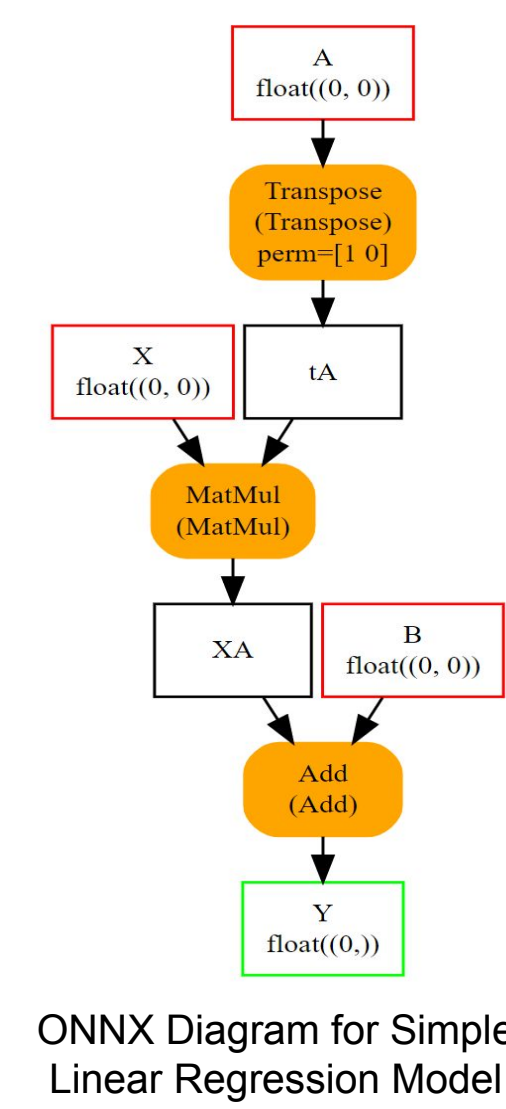
Context

Open Neural Network Exchange (**ONNX**)

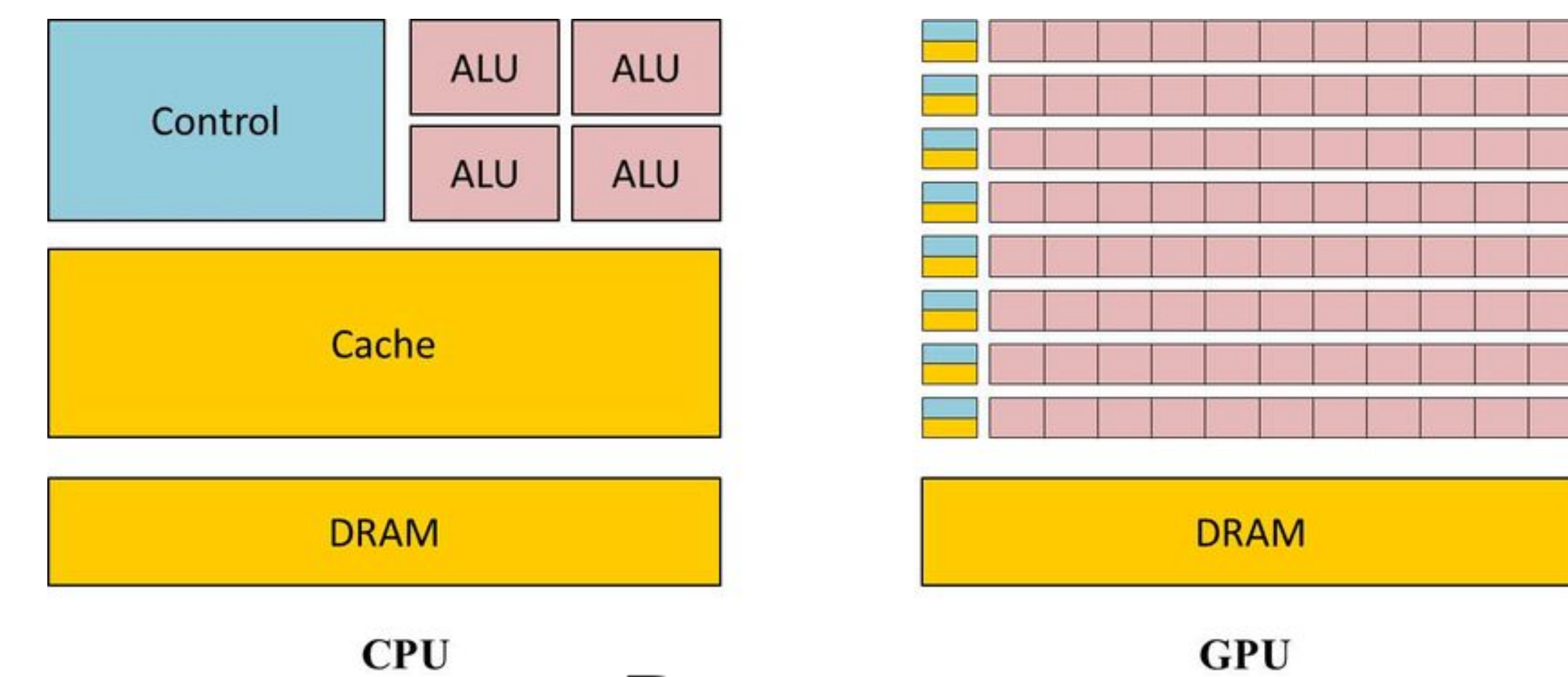
- Open-source, framework-agnostic format for representing machine learning models.
- Encodes models as graphs composed of operator nodes and tensor initializers.
- Designed for interoperability and easy model exchange across frameworks

Model Used in Our Program

- Our program uses the **SMOLLM2-135M** model

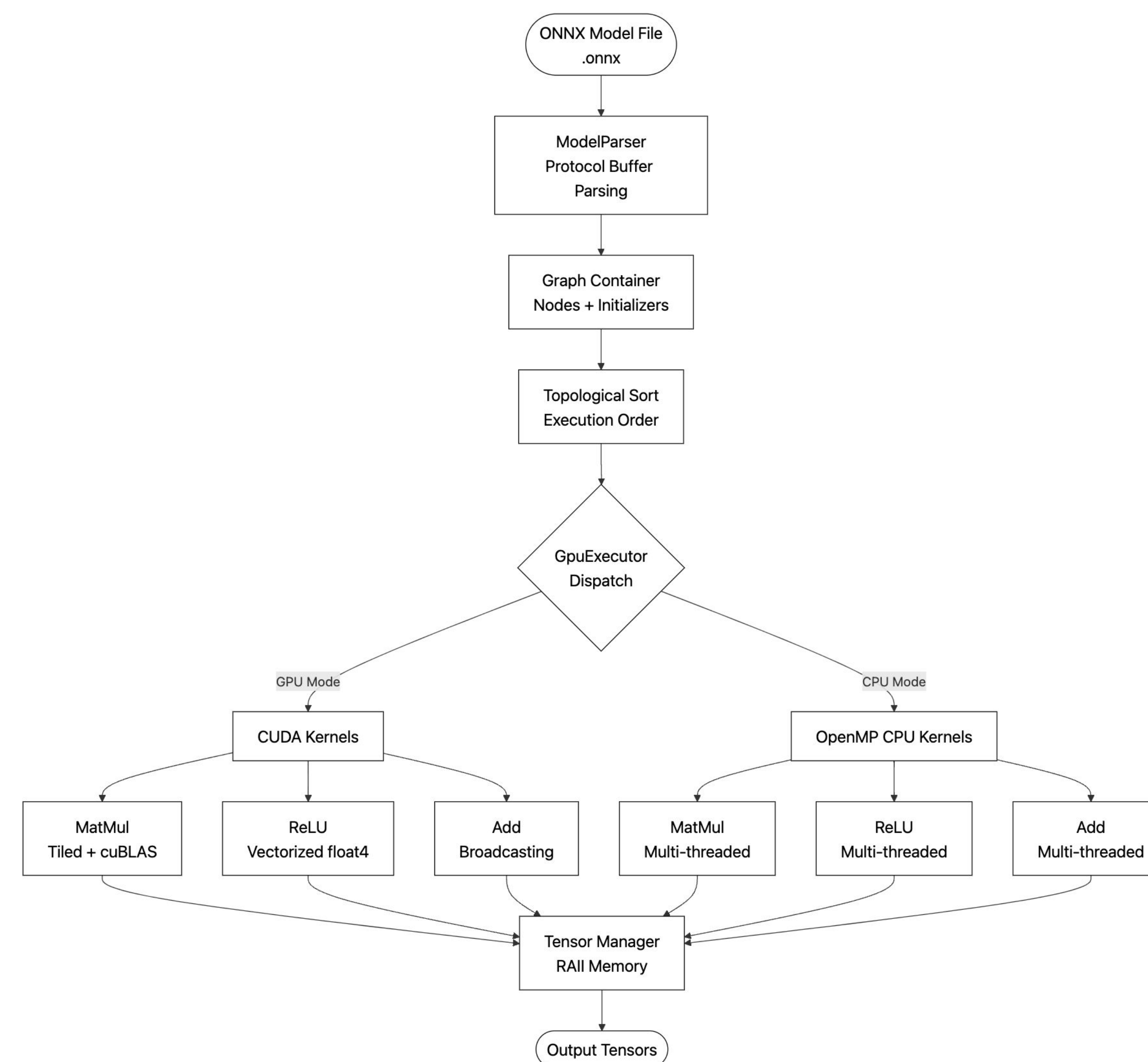


→ CPU vs. GPU



Architecture

- Parses **ONNX** model into an internal graph and determines execution order via topological sort
- Dispatches each operator to **CUDA kernels**
- Uses optimized GPU primitives (tiled MatMul, vectorized ReLU, broadcasting Add)
- Manages all host/GPU buffers through a unified **RAII-based Tensor Manager** for safe, efficient memory handling



Optimizations

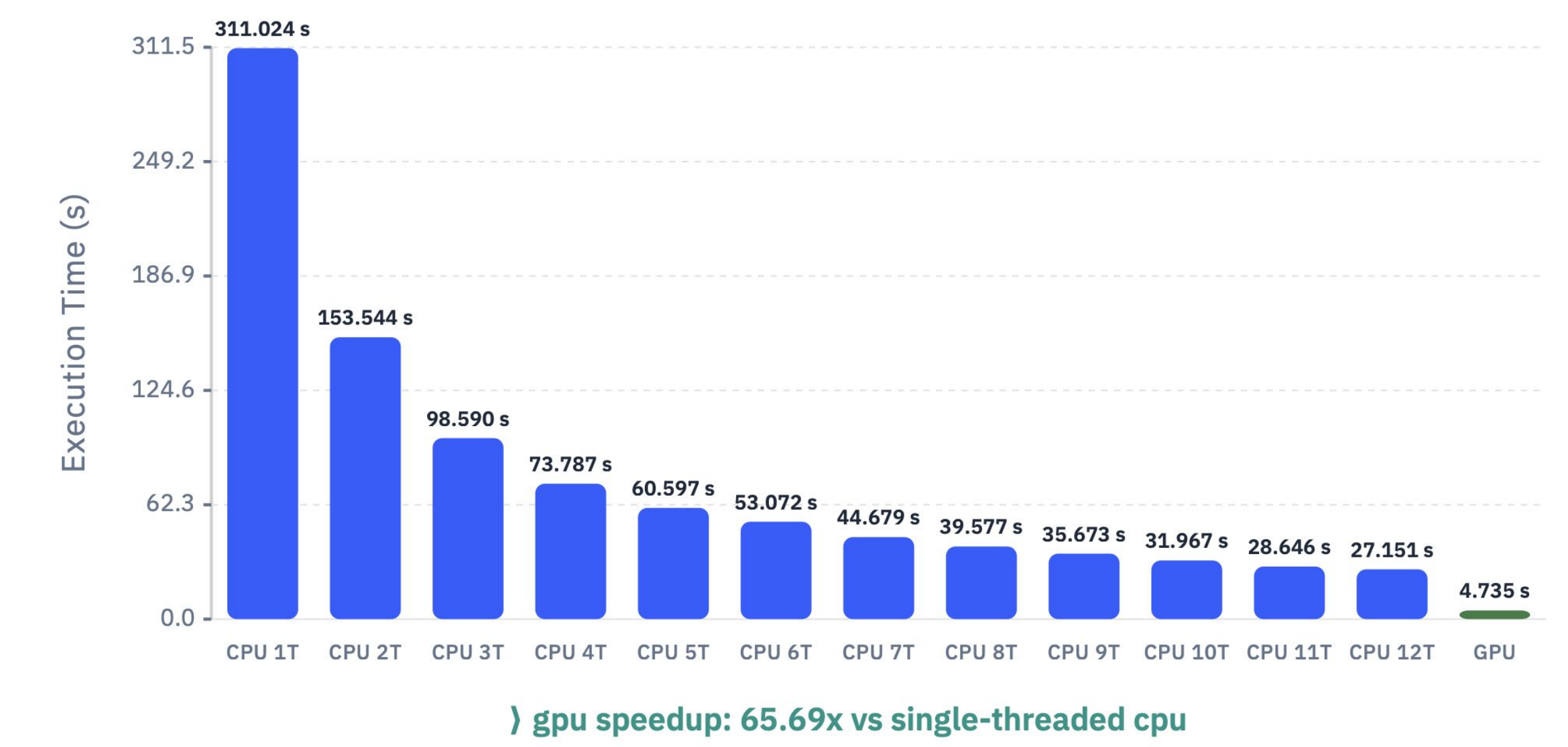
KV Caching (Key-Value Caching)

- Reuses previously computed attention key/value tensors
- Reduces autoregressive decoding complexity from $O(N^2)$ to $O(N)$.

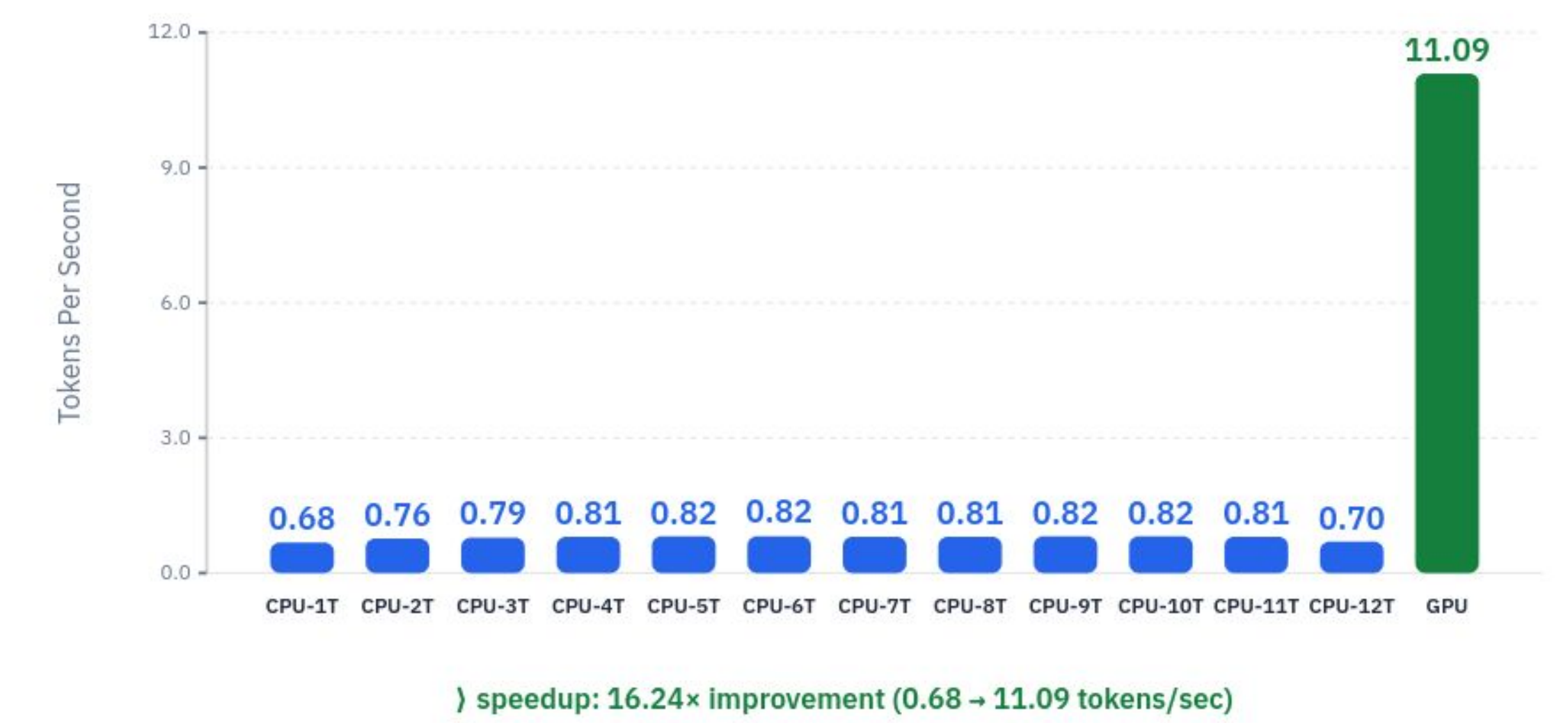
GPU Execution

- Moved all model operations (MatMul, Attention, LayerNorm, etc.) onto the GPU.
- Eliminated CPU $\leftrightarrow$ GPU memory transfer overhead during inference.

Results



→ On our largest MLP (~500 million parameters) we achieved a **65.69x speedup** on the GPU



→ Achieved **11.09 tokens per second** on GPU, a 16.24x speedup vs CPU

TODO: Power locally vs Data center

Future Improvements

Ability to run more **LLMs**

- Need to implement more kernels
- Additional activation functions (e.g. GELU, SiLU)

More Optimizations

- **Quantization** support (INT8/FP8)